Kế hoạch giải hệ tuyến tính song song trên GPU

Môn Lập trình tính toán song song

Nhóm [Tên nhóm và thành viên]

1 Phạm vi và câu hỏi thí nghiệm

Nhóm xây dựng bản tuần tự trên CPU và bản song song trên GPU cho **Blocked Cholesky**, **Jacobi** và **Conjugate Gradient (CG)**. Ba thuật toán giải cùng hệ $Ax = b$ với A là ma trận thực, đặc, đối xứng xác định dương (SPD) và được lưu tường minh. Bộ đo chính dùng một vế phải mỗi lượt, cùng dữ liệu và cùng độ chính xác số cho hai nền tảng. Bài tích vô hướng hai vector là sản phẩm chung của môn học và được kiểm thử riêng. Mã reduction của bài chung có thể dùng lại trong CG sau khi kiểm chứng.

- Với mỗi thuật toán, GPU giảm thời gian giải bao nhiêu so với CPU một luồng? Kích thước nào bắt đầu bù được chi phí truyền dữ liệu?
- Cholesky nhạy ra sao với kích thước tile? Với Jacobi và CG, phép nhân ma trận–vector, reduction và đồng bộ chiếm bao nhiêu?
- Thay đổi tần suất kiểm tra dừng của Jacobi ảnh hưởng ra sao đến số vòng lặp, độ chính xác và tổng thời gian? Hai vùng nhớ nghiệm của Jacobi đóng góp bao nhiêu vào bộ nhớ và thời gian xử lý?

2 Bộ ma trận và kiểm thử

Bộ ma trận SPD có thể lấy từ nguồn bên ngoài hoặc tự sinh. Nhóm sẽ chốt nguồn, cách tái tạo và cách kiểm tra tính phù hợp với cả ba bộ giải, bao gồm điều kiện hội tụ của Jacobi. CPU và GPU dùng cùng A, b.

[Nguồn dữ liệu hoặc cách sinh ma trận]

Chọn cách xác định nghiệm tham chiếu $x^\star$. Dùng cùng $x^{(0)}$ cho Jacobi và CG. Bộ kiểm thử có ca nhỏ biết nghiệm, kích thước dư tile/block và trường hợp đạt giới hạn vòng lặp. Kiểm tra độc lập nghiệm cuối từ A, b gốc. Dùng công cụ kiểm tra truy cập và đồng bộ nếu backend hỗ trợ.

[Cách xác định nghiệm tham chiếu]

3 Thiết kế song song theo PCAM

3.1 Blocked Cholesky

P — Phân hoạch (Partitioning) [Điền cách chia công việc và dữ liệu]

C — Giao tiếp (Communication) [Điền dữ liệu trao đổi và điểm đồng bộ]

A — Gộp (Agglomeration) [Điền cách gộp các đơn vị công việc]

M — Ánh xạ (Mapping) [Điền cách gán công việc lên GPU]

3.2 Jacobi

P — Phân hoạch (Partitioning) [Điền cách chia công việc và dữ liệu]

C — Giao tiếp (Communication) [Điền dữ liệu trao đổi và điểm đồng bộ]

A — Gộp (Agglomeration) [Điền cách gộp các đơn vị công việc]

M — Ánh xạ (Mapping) [Điền cách gán công việc lên GPU]

3.3 Conjugate Gradient

P — Phân hoạch (Partitioning) [Điền cách chia công việc và dữ liệu]

C — Giao tiếp (Communication) [Điền dữ liệu trao đổi và điểm đồng bộ]

A — Gộp (Agglomeration) [Điền cách gộp các đơn vị công việc]

M — Ánh xạ (Mapping) [Điền cách gán công việc lên GPU]

4 Các mốc thực hiện

1. Tạo kho Git trong hai tuần đầu. Chốt môi trường, dạng đầu vào, lịch và người phụ trách. Làm bài tích vô hướng, chuẩn bị dữ liệu, bộ kiểm thử nhỏ và khung ghi kết quả.
2. Viết ba bản CPU một luồng. Kiểm tra sai số và trạng thái trả về. Ghi thứ tự lưu dữ liệu, lựa chọn thư viện và cờ biên dịch.
3. Viết ba bản GPU. Đối chiếu CPU/GPU trên bộ nhỏ, kiểm tra kích thước dư và đo thử từng pha trước khi chạy benchmark chính.
4. Chạy bộ đo đã khóa cấu hình, giữ dữ liệu thô và log lỗi. Phân tích độ phân tán, thời gian giải, hội tụ và chi phí đồng bộ.
5. Hoàn thiện mã, script build/chạy/vẽ, dữ liệu mẫu hoặc script sinh, README tái lập, báo cáo, slide, `AI_USAGE.md` và `CONTRIBUTIONS.md`. Mỗi thành viên dùng tài khoản riêng và có commit thể hiện đóng góp. Ghi nguồn và giấy phép mã ngoài.

[Người phụ trách từng mốc] [Hạn từng mốc]

5 Ma trận cấu hình thí nghiệm

Trục thay đổi	Cấu hình và đối chứng
Dữ liệu	Ít nhất ba kích thước $n_1 < n_2 < n_3$, trong đó ca lớn đủ để đánh giá GPU và vừa ngân sách bộ nhớ của cả ba bộ giải. Mỗi trường hợp giữ nguyên ma trận và vế phải khi đối chiếu. [Kích thước và mã các bộ ma trận]
Nền tảng	CPU một luồng và GPU trên cùng bài toán. Ghi model CPU/GPU, số lõi, RAM/VRAM, hệ điều hành, backend, driver, compiler, cờ tối ưu, phiên bản thư viện và số luồng thư viện CPU. [Máy chạy, backend và kiểu số]
Blocked Cholesky	Quét kích thước tile tại các n đại diện. Giữ cấu hình còn lại cố định và ghi cách xử lý tile dư. [Tile size và kích thước quét]
Jacobi	Đối chiếu kiểm tra phần dư mỗi vòng với mỗi q vòng. Tách phép đo đổi con trỏ và sao chép vector thành thí nghiệm riêng, giữ nguyên q. Quét cấu hình nhóm luồng của kernel cập nhật nếu cần. [Các giá trị q và threads/block]
CG	Ghi cấu hình kernel Ap và reduction, vị trí lưu vô hướng, và quy tắc kiểm tra dừng. Khi quét số luồng mỗi block, giữ thuật toán reduction, tiêu chuẩn dừng và dữ liệu cố định. [Cấu hình kernel và reduction]
Chạy lặp	Chạy nóng trước, sau đó lấy tối thiểu năm lượt đo mỗi cấu hình. Khôi phục A nếu bị ghi đè và đặt lại trạng thái nghiệm trước mỗi lượt. Đo cùng ranh giới pha và ghi rõ cấu hình chạy bị lỗi. [Số warm-up, số lượt đo và thứ tự chạy]

5.1 Các quyết định cần khóa trước khi chạy

- Chọn FP32 hoặc FP64 cho bộ đo chính và khóa dung sai phù hợp.
  [Dung sai phần dư, ngưỡng sai số và số vòng tối đa]
- Chốt số GPU và cách khảo sát tài nguyên. Thay đổi threads/block trên cùng GPU là khảo sát cách ánh xạ. Nếu có p GPU tương đương và bản chạy đa GPU, cố định n để đo strong scaling. [Số GPU khả dụng và trục đo efficiency]
- Chọn chính sách dùng thư viện GPU, chế độ cấp phát và phạm vi profiling.
  [Thư viện hoặc kernel tự viết]

6 Chỉ số và cách đo

6.1 Độ chính xác và hội tụ

Gọi $r^{(k)} = b - Ax^{(k)}$ là vector phần dư và $e^{(k)} = x^{(k)} - x^\star$ là vector sai số. Nhóm chọn bao nhiêu chuẩn tùy mục tiêu thí nghiệm và ghi riêng chuẩn dùng để quyết định dừng.

[Chọn số lượng và danh sách chuẩn cần báo cáo]

[Chọn chuẩn dừng, ngưỡng và cách kết hợp điều kiện]

Với mỗi chuẩn $\|\cdot\|_\nu$ đã chọn, tính

$$R_\nu^{(k)} = \|r^{(k)}\|_\nu, \qquad R_{\nu,\mathrm{rel}}^{(k)} = \frac{R_\nu^{(k)}}{\|b\|_\nu},$$

$$E_\nu^{(k)} = \|e^{(k)}\|_\nu, \qquad E_{\nu,\mathrm{rel}}^{(k)} = \frac{E_\nu^{(k)}}{\|x^\star\|_\nu}.$$

Khi mẫu số bằng không, báo chỉ số tuyệt đối tương ứng. Ví dụ về công thức chuẩn cho vector $v \in \mathbb{R}^n$:

$$\|v\|_1 = \sum_{i=1}^{n} |v_i|, \qquad \|v\|_2 = \left(\sum_{i=1}^{n} |v_i|^2\right)^{1/2}, \qquad \|v\|_\infty = \max_{1 \le i \le n} |v_i|.$$

Gọi $\mathcal{K}$ là tập các vòng được kiểm tra, $D(k)$ là điều kiện dừng đã chọn và $K_{\max}$ là giới hạn vòng lặp. Ví dụ, $D(k) = [R_{\nu,\mathrm{rel}}^{(k)} \le \varepsilon_\nu]$. Nếu có vòng đạt chuẩn, số vòng đến hội tụ và số lần kiểm tra đến lúc dừng là

$$K_{\mathrm{conv}} = \min\{k \in \mathcal{K} : D(k)\}, \qquad N_{\mathrm{check}} = \big|\{k \in \mathcal{K} : k \le K_{\mathrm{stop}}\}\big|,$$

với K_{stop} là vòng thực tế dừng. Giá trị này bằng K_{conv} khi đạt chuẩn hoặc $K_{\max}$ khi chạm giới hạn. Ghi riêng lỗi số học. Với Jacobi, so sánh $\mathcal{K}$ khi kiểm tra mỗi vòng và mỗi q vòng, bao gồm lần kiểm ở giới hạn nếu vòng đó chưa được kiểm. Với CG, có thể theo dõi phần dư truy hồi trong vòng lặp và xác nhận nghiệm cuối bằng $b - Ax$ tính lại. Cholesky chỉ dùng phép kiểm cuối. Báo số vòng thực tế và lý do dừng.

6.2 Tổng hợp các lượt đo

Với m thời gian hợp lệ $t_1, \ldots, t_m$ cho cùng cấu hình và phạm vi, tính

$$\widetilde{T} = \mathrm{median}(t_1, \ldots, t_m), \qquad [T_{\min}, T_{\max}] = [\min_i t_i, \max_i t_i], \qquad \mathrm{IQR} = Q_{0.75} - Q_{0.25}.$$

Q_a là phân vị mức a. Báo median và ít nhất một trong hai chỉ số phân tán. Ghi m và loại lượt warm-up hoặc profiling ra khỏi tập thời gian so sánh chính. Giữ cả bản ghi các lượt thất bại.

6.3 Thời gian, đồng bộ và bộ nhớ

Đo wall-clock từ đầu thiết lập hoặc nạp đầu vào đến khi nghiệm về host. Ghi riêng thiết lập, I/O đầu vào, H2D ban đầu, pha giải và D2H cuối. Thời gian giải gồm truyền vô hướng và chờ phát sinh trong vòng lặp. Kiểm chứng nghiệm sau giải và ghi log nằm ngoài thời gian tổng.

$$T_{\text{total}} = t^{\text{end}}_{\text{total}} - t^{\text{start}}_{\text{total}}, \qquad T_{\text{solve}} = t^{\text{end}}_{\text{solve}} - t^{\text{start}}_{\text{solve}}.$$

Các mốc t ở đây dùng cùng đồng hồ host và quy đổi sang ms. Dùng event GPU để đo hoạt động thiết bị. Với pha ϕ có nhiều khoảng đo, tính

$$T_\phi = \sum_j (t^{\text{end}}_{\phi,j} - t^{\text{start}}_{\phi,j}), \qquad P_\phi = 100 \frac{T_\phi}{T_{\text{solve}}}\ \%.$$

Đo riêng phân tích, giải tiến và giải lùi cho Cholesky. Đối với Jacobi và CG, tách cập nhật, nhân ma trận–vector, reduction, chờ đồng bộ và truyền vô hướng theo khả năng đo. Các pha chồng lấp có thể cho tổng tỷ lệ trên 100%. Đo profiling ở lượt riêng nếu công cụ ảnh hưởng thời gian.

Gọi N_ϕ là số lần phát sinh hoạt động ϕ, chẳng hạn số lần reduction, chờ host hoặc sao chép vector. Với $K_{\text{stop}} > 0$,

$$N_\phi = \sum_j \mathbf{1}[\text{hoạt động } j \text{ thuộc pha } \phi], \qquad \overline{T}_{\text{iter}} = \frac{T_{\text{solve}}}{K_{\text{stop}}}, \qquad \overline{N}_\phi = \frac{N_\phi}{K_{\text{stop}}}.$$

Đối với Jacobi, ghi bộ nhớ dành cho hai vector, đỉnh bộ nhớ thiết bị đang dùng và lượng dữ liệu sao chép thiết bị–thiết bị.

$$M_{\text{vectors}} = 2ns, \qquad M_{\text{peak}} = \max_t M_{\text{device}}(t), \qquad B_{\text{copy}} = \sum_{j=1}^{N_{\text{copy}}} b_j.$$

s là số byte mỗi phần tử và b_j là số byte truyền ở lần sao chép thứ j. Với sao chép toàn bộ vector, $b_j = ns$. Ghi T_{copy} theo công thức thời gian pha ở trên.

6.4 Speedup và efficiency

So sánh cùng thuật toán, đầu vào, độ chính xác và phạm vi thời gian.

$$S_{\text{GPU}} = \frac{\widetilde{T}_{\text{CPU},1}}{\widetilde{T}_{\text{GPU}}}, \qquad S_p = \frac{\widetilde{T}_1}{\widetilde{T}_p}, \qquad E_p = \frac{S_p}{p}.$$

Báo speedup riêng cho thời gian tổng và pha giải. Dùng E_p khi p đếm các đơn vị tài nguyên đồng nhất, chẳng hạn số GPU cùng loại. Nếu chỉ có một GPU, báo ảnh hưởng của threads/block và chốt cách báo cáo efficiency với giảng viên. Khi xếp hạng các bộ giải, dùng thời gian đạt cùng tiêu chuẩn nghiệm và tính cả pha phân tích Cholesky.

7 Bản ghi của một lượt chạy

Đề xuất JSON Lines cho dữ liệu thô, một đối tượng mỗi lượt. Có thể xuất CSV phẳng để vẽ biểu đồ. Các trường không áp dụng dùng `null`. Một cấu hình có nhiều `run_id` và cùng `config_id`. `env_id` trỏ tới bản mô tả máy và phần mềm lưu cùng dữ liệu thí nghiệm.

Nhóm trường	Thông tin cần ghi
Định danh	`run_id`, `config_id`, `env_id`, `timestamp`, `repeat_index`, `warmup`, `profiled`, `git_commit`.
Đầu vào	`matrix_id`, `n`, `source_ref`, `seed`, `matrix_family`, `rhs_id`, `x0_id`, `dtype`. Lưu mã phiên bản bộ sinh hoặc checksum trong manifest dữ liệu.
Biến thể	`solver`, `backend`, `cpu_threads`, `gpu_count`, `tile_size`, `threads_per_block`, `reduction_variant`, `buffer_mode`, `stop_rule`, `check_every`, `report_norms`, `stop_thresholds`, `error_thresholds`, `max_iters`.
Thời gian	`total_ms`, `setup_ms`, `io_ms`, `h2d_ms`, `solve_ms`, `d2h_ms`. Nhóm `phase_ms` chứa `factor`, `forward`, `backward`, `matvec`, `update`, `reduction`, `sync_wait`, `scalar_transfer`, `buffer_copy`, tùy thuật toán và phương pháp đo.
Tài nguyên	`peak_device_bytes`, `jacobi_vector_bytes`, `buffer_copy_count`, `buffer_copy_bytes`, `reduction_count`, `host_sync_count`.
Kết quả	`status`, `stop_reason`, `iterations`, `checks`, `norm_metrics`, `log_ref`. `norm_metrics` lưu phần dư và sai số (tuyệt đối, tương đối) theo mã từng chuẩn.

Mỗi cấu hình lưu lệnh chạy, tham số, thông tin môi trường và phương pháp đo. Từ dữ liệu thô, tạo bảng median và độ phân tán, đồ thị thời gian/speedup theo n, biểu đồ phân bổ thời gian theo pha và đồ thị phần dư theo vòng lặp từ các lượt đo hội tụ riêng.

8 Các phần mở rộng tùy chọn

Sau khi có số liệu cho bộ đo chính, có thể chọn một nhánh riêng. **Sparse** dùng CSR cho Jacobi/CG và báo kích thước, số phần tử khác không và băng thông. **Matrix-free** đo CG với một toán tử đại số có cấu trúc, đối chiếu cùng A lưu tường minh. **Tiền điều kiện** đối chiếu CG và PCG với $M = \mathrm{diag}(A)$, gồm chi phí tạo và áp dụng M^{-1}, số vòng lặp và thời gian đạt chuẩn. Mỗi nhánh có bộ so sánh riêng.

Căn cứ yêu cầu môn học. Chu de NHOM 2026-1 v1s.docx, mục A1, B, B.1, B.2 và Chủ đề 1.